

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 18

Dynamic Programming

Overview

Last time

- Hashing and hash tables: pros and cons
- Collisions: separate chaining and open-addressing
- Load factor: space vs. efficiency

Today

- Dynamic Programming

Dynamic Programming

Dynamic programming

- algorithm design technique
- recurrence relation
- recursion involves overlapping instances

Dynamic Programming

Dynamic programming

- algorithm design technique
- recurrence relation
- recursion involves overlapping instances

Simple instance: Fibonacci numbers

- rather than naive top-down algorithm
- solve and tabulate smaller sub-problems

Rather than solve overlapping sub-problems repeatedly

- bottom-up approach
- use tabulated results

Dynamic Programming and Optimization

Optimization problems

- sometimes allow for clever dynamic programming solutions

Prerequisite: certain preservation of optimality

- an optimal solution to an instance obtained by
- combining optimal solutions to the sub-instances

Example 1: The Coin-Row Problem

Given a row of coins, pick up the largest possible sum.
Constraint: No two adjacent coins can be picked.

Example 1: The Coin-Row Problem

Given a row of coins, pick up the largest possible sum.
Constraint: No two adjacent coins can be picked.

Think of the problem recursively.

Example 1: The Coin-Row Problem

Given a row of coins, pick up the largest possible sum.
Constraint: No two adjacent coins can be picked.

Think of the problem recursively.

Let the values of the coins be $v_1, v_2, \ldots v_n$.

Let $S(i)$ be the sum that can be gotten by picking optimally from the first i coins.

Either the ith coin (with value v_i) is part of the solution or it is not.

If we choose to pick it up then we cannot also pick its neighbour on the left, so the best we can achieve is $S(i-2) + v_i$.

Otherwise we leave it, and the best we can achieve is $S(i-1)$.

Example 1: The Coin-Row Problem

Formal Recurrence Relation

$$S(n) = max(S(n-1), S(n-2) + v_n)$$

for $n > 1$.

And two base cases: $S(0) = 0$ and $S(1) = v_1$

Example 1: The Coin-Row Problem

Formal Recurrence Relation

$$S(n) = max(S(n-1), S(n-2) + v_n)$$

for $n > 1$.

And two base cases: $S(0) = 0$ and $S(1) = v_1$

If directly coded, as above

- same problem as the naive Fibonacci program:
- lots of repetition of identical sub-computations.

Example 1: The Coin-Row Problem

Observation

- We need all values $S(1)$ to $S(n)$ anyway
- So: compute them bottom-up
- And remember in an array S

Given an array C of coin values:

function CoinRow($C[\cdot], n$)
    $S[0] \leftarrow 0$
    $S[1] \leftarrow C[1]$
    for $i \leftarrow 2$ to n **do**
        $S[i] \leftarrow max(S[i-1], S[i-2] + C[i])$
    return $S[n]$

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20						

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20					

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20	40				

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20	40	70			

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20	40	70	70		

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20	40	70	70	80	

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20	40	70	70	80	90

Example 1: The Coin-Row Problem

Let us run the algorithm on the coins 20, 10, 20, 50, 20, 10, 20:

index:	0	1	2	3	4	5	6	7
C :		20	10	20	50	20	10	20
S :	0	20	20	40	70	70	80	90
$Using$:		1	1	1	1	1	1	1
				3	4	4	4	4
							6	7

Finding optimal solution: indices of the coins used

- easy extension of the algorithm

Example 2: The Knapsack Problem

(Already investigated in Lecture 5)

Example 2: The Knapsack Problem

(Already investigated in Lecture 5)

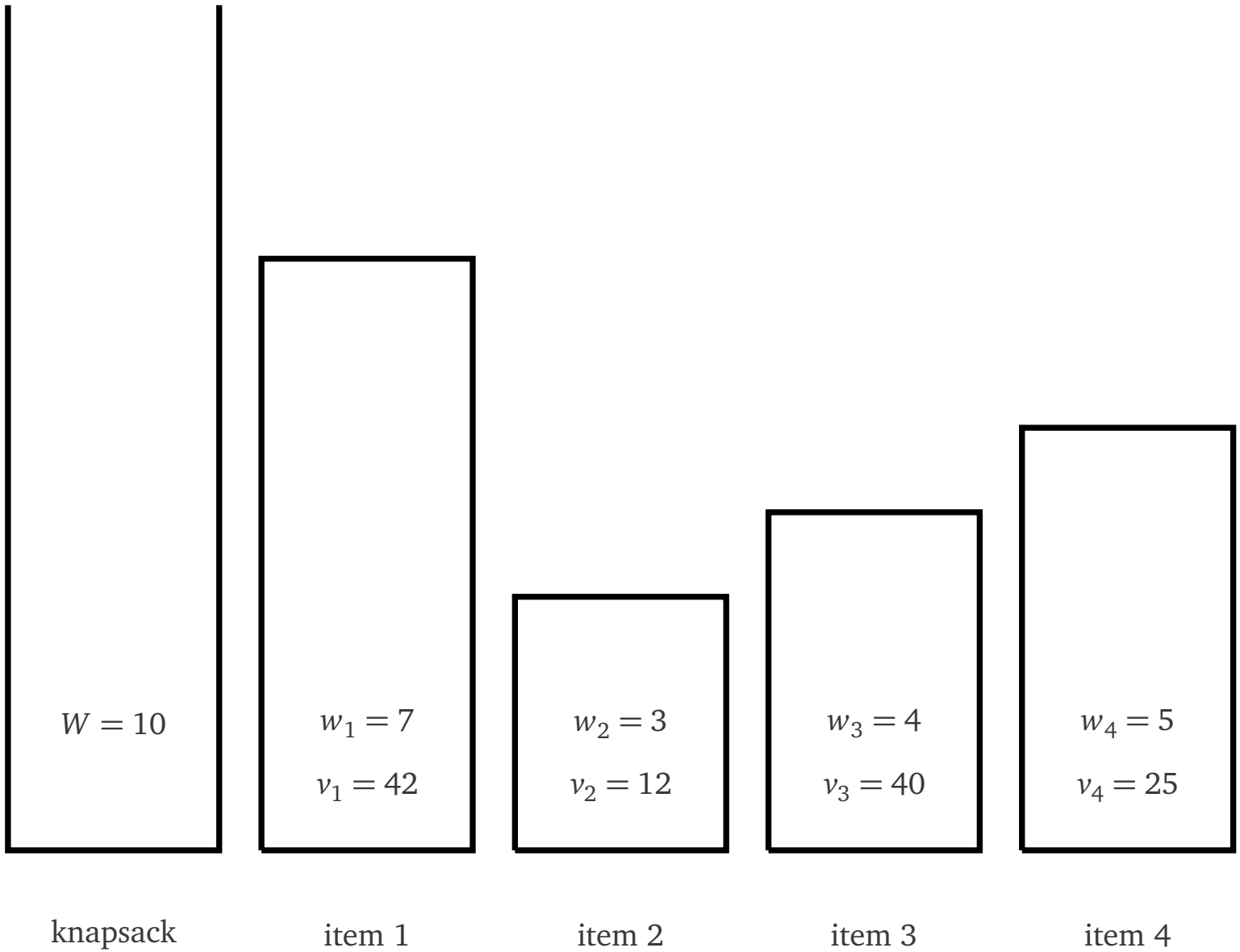
Given n items with

- weights: $w_1, w_2, \ldots, w_n$
- values: $v_1, v_2, \ldots, v_n$
- knapsack of capacity W

find the most valuable selection of items that will fit in the knapsack.

We assume that all entities involved are positive integers.

Example 2: Knapsack



Example 2: The Knapsack Problem

- Rather than brute force, lets try dynamic programming
- Critical step: “what is the sub-problem?”

Example 2: The Knapsack Problem

- Rather than brute force, lets try dynamic programming
- Critical step: "what is the sub-problem?"

Formulate a recurrence relation over two parameters:

- sequence $1, 2, \ldots i$ of items considered so far
- remaining capacity $w \leq W$.

Example 2: The Knapsack Problem

- Rather than brute force, lets try dynamic programming
- Critical step: "what is the sub-problem?"

Formulate a recurrence relation over two parameters:

- sequence $1, 2, \ldots i$ of items considered so far
- remaining capacity $w \leq W$.

Let $K(i, w)$ be the value of the best choice of items amongst the first i using knapsack capacity w.

Find $K(n, W)$.

Example 2: The Knapsack Problem

Why $K(i, w)$

- Because we can express a solution to that recursively.
  Amongst the first i items we either pick item i or we don't.

Example 2: The Knapsack Problem

Why $K(i, w)$

- Because we can express a solution to that recursively.
  Amongst the first i items we either pick item i or we don't.

Exclude item i

- the value of an optimal subset is simply $K(i-1, w)$

Example 2: The Knapsack Problem

Why $K(i, w)$

- Because we can express a solution to that recursively. Amongst the first i items we either pick item i or we don't.

Exclude item i

- the value of an optimal subset is simply $K(i-1, w)$

Include item i

- optimal solution contains an optimal subset of the first $i-1$ items
- that will fit into a bag of capacity $w - w_i$
- With the item i, this is $K(i-1, w-w_i) + v_i$.

Example 2: The Knapsack Problem

Why $K(i, w)$

- Because we can express a solution to that recursively. Amongst the first i items we either pick item i or we don't.

Exclude item i

- the value of an optimal subset is simply $K(i-1, w)$

Include item i

- optimal solution contains an optimal subset of the first $i-1$ items
- that will fit into a bag of capacity $w - w_i$
- With the item i, this is $K(i-1, w-w_i) + v_i$.

...provided item i fits, that is, provided $w - w_i \geq 0$.

Example 2: The Knapsack Problem

Now it is easy to express the solution recursively:

$$K(i, w) = 0 \text{ if } i = 0 \text{ or } w = 0$$

Otherwise:

$$K(i, w) = \begin{cases} max(K(i-1, w), K(i-1, w-w_i) + v_i) & \text{if } w \geq w_i \\ K(i-1, w) & \text{if } w < w_i \end{cases}$$

Example 2: The Knapsack Problem

Now it is easy to express the solution recursively:

$$K(i,w) = 0 \text{ if } i = 0 \text{ or } w = 0$$

Otherwise:

$$K(i,w) = \begin{cases} max(K(i-1,w), K(i-1,w-w_i) + v_i) & \text{if } w \geq w_i \\ K(i-1,w) & \text{if } w < w_i \end{cases}$$

Correct algorithm, but possibly inefficient

- Directly implemented top-down: very inefficient
- Bottom-up: fill a table will have $n+1$ rows and $W+1$ columns

Example 2: The Knapsack Problem

First fill leftmost column and top row, then proceed row by row:

for $i \leftarrow 0$ to n **do**
    $K[i, 0] \leftarrow 0$
for $j \leftarrow 1$ to W **do**
    $K[0, j] \leftarrow 0$
for $i \leftarrow 1$ to n **do**
    for $j \leftarrow 1$ to W **do**
        if $j < w_i$ **then**
            $K[i, j] \leftarrow K[i-1, j]$
        else
            $K[i, j] \leftarrow max(K[i-1, j], K[i-1, j-w_i] + v_i)$
return $K[n, W]$

The algorithm has time (and space) complexity .

Example 2: The Knapsack Problem

First fill leftmost column and top row, then proceed row by row:

for $i \leftarrow 0$ to n **do**
    $K[i, 0] \leftarrow 0$
for $j \leftarrow 1$ to W **do**
    $K[0, j] \leftarrow 0$
for $i \leftarrow 1$ to n **do**
    for $j \leftarrow 1$ to W **do**
        if $j < w_i$ **then**
            $K[i, j] \leftarrow K[i-1, j]$
        else
            $K[i, j] \leftarrow max(K[i-1, j], K[i-1, j-w_i] + v_i)$
return $K[n, W]$

The algorithm has time (and space) complexity $\Theta(nW)$.

Example 2: The Knapsack Problem

Here is a concrete example—like before, except we let $W = 8$.

	$i \backslash j$	0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0	0
$v_1 = 42, w_1 = 7$	1	0	0	0	0	0	0	0		
$v_2 = 12, w_2 = 3$	2	0	0	0						
$v_3 = 40, w_3 = 4$	3	0	0	0						
$v_4 = 25, w_4 = 5$	4	0								

Example 2: The Knapsack Problem

Here is a concrete example—like before, except we let $W = 8$.

	$i \backslash j$	0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0	0
$v_1 = 42, w_1 = 7$	1	0	0	0	0	0	0	0	42	42
$v_2 = 12, w_2 = 3$	2	0	0	0	12	12	12	12	42	42
$v_3 = 40, w_3 = 4$	3	0	0	0	12	40	40	40		
$v_4 = 25, w_4 = 5$	4	0								

Example 2: The Knapsack Problem

Here is a concrete example—like before, except we let $W = 8$.

	$i \backslash j$	0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0	0
$v_1 = 42, w_1 = 7$	1	0	0	0	0	0	0	0	42	42
$v_2 = 12, w_2 = 3$	2	0	0	0	12	12	12	12	42	42
$v_3 = 40, w_3 = 4$	3	0	0	0	12	40	40	40		
$v_4 = 25, w_4 = 5$	4	0								

Example 2: The Knapsack Problem

Here is a concrete example—like before, except we let $W = 8$.

	$i \backslash j$	0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0	0
$v_1 = 42, w_1 = 7$	1	0	0	0	0	0	0	0	42	42
$v_2 = 12, w_2 = 3$	2	0	0	0	12	12	12	12	42	42
$v_3 = 40, w_3 = 4$	3	0	0	0	12	40	40	40	52	52
$v_4 = 25, w_4 = 5$	4	0	0	0	12	40	40	40	52	52

- optimal value is 52
- optimal combination: follow the computation

Solving the Knapsack Problem with Memoing

bottom-up (table-filling) solution is overkill

Solving the Knapsack Problem with Memoing

bottom-up (table-filling) solution is overkill

- It finds the solution to every conceivable sub-instance

Most table entries cannot contribute to a solution

- consider all the weights in the problem be multiplied by 1000

Solving the Knapsack Problem with Memoing

bottom-up (table-filling) solution is overkill

- It finds the solution to every conceivable sub-instance

Most table entries cannot contribute to a solution

- consider all the weights in the problem be multiplied by 1000

Top-down with memoing is preferable

- To keep the memo table small: make it a hash table.

Solving the Knapsack Problem with Memoing

The hash table uses keys (i, j).
Stored together with their corresponding values $k = K(i, j)$.

function KNAP(i, j)
  if $i = 0$ or $j = 0$ **then**
    return 0
  if key (i, j) is in hashtable **then**
    return the corresponding value (that is, $K(i, j)$)
  if $j < w_i$ **then**
    $k \leftarrow$ KNAP$(i - 1, j)$
  else
    $k \leftarrow max($KNAP$(i - 1, j),$ KNAP$(i - 1, j - w_i) + v_i)$
  insert (i, j) and k into hashtable, with key (i, j)
  return k

Edit Distance

Spelling checkers usually suggest alternatives for words they do not recognize, trying to find words that are close in some sense.

One measure of closeness between two words is how well the align, when we allow for gaps (missing characters).

```
S M I _ L E     _ S M I L E     S _ M I L E
S I M I L E     S I M I L E     S I M I L E
```

Let us say the cost of an alignment is the number of places where two (filled) strings differ.

In the example, the costs are 3, 2, and 1.

The edit distance is the cost of the best possible alignment.

Finding Edit Distances with Dynamic Programming

Consider edit distances between prefixes of the given strings s and t: the first i characters of x and the first j characters of t.

Summary

Today

Dynamic Programming

Next

- Dynamic programming applied to graph problems
- Transitive closure, Shortest-paths
- Warshall's and Floyd's algorithms.